Запознавање со Spring рамката

Веб програмирање

Универзитет „Св. Кирил и Методиј“ во Скопје
ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

Преглед на темата

Зависности и DI

- Зошто Spring?
- Што е зависност
- Проблемот со `new`
- DI без Spring
- DIP, DI и IoC

Бинови и Spring Boot

- IoC контејнер
- Што е бин
- `@Component`, `@Service`
- Прва Spring Boot апликација

DI во пракса

- `@Autowired` и видови вметнување
- Слоеви на апликацијата
- `@Primary`, `@Qualifier`
- `@Bean` и `@Configuration`

Конфигурација и тестови

- Singleton и опсези
- Животен циклус
- `application.properties`
- `@Value`, `@Profile`
- Тестирање со DI

Еден пример низ целата тема: **NotificationController → NotificationService → NotificationSender**

Зошто Spring?

Зависности и вметнување на зависности (без Spring)

Цели

1. Да објасниме **што е Spring** и зошто се користи
2. Да разбереме што е **зависност** помеѓу две класи
3. Да го препознаеме проблемот со new во класата (**цврсто поврзување**)
4. Да примениме **вметнување на зависности (DI)** преку конструктор — без Spring
5. Да ги разликуваме поимите **DIP**, **DI** и **IoC**

Што е Spring?

- **Рамка (framework)** за развој на Java апликации
- Ги креира и поврзува објектите (компонентите) на апликацијата
- Нуди готова инфраструктура: веб (HTTP), бази на податоци, безбедност, тестирање
- Ние пишуваме **бизнис логика**, Spring се грижи за поврзувањето
- Отворен код, голема заедница, индустриски стандард

Библиотека или рамка?

Библиотека

Ти ја повикуваш кога ти треба

```
Math.max(a, b)
```

Рамка

Таа го повикува твојот код

Spring го повикува твојот контролер кога ќе пристигне HTTP барање

Каде застанавме: сервлети

Со сервлети (досега)

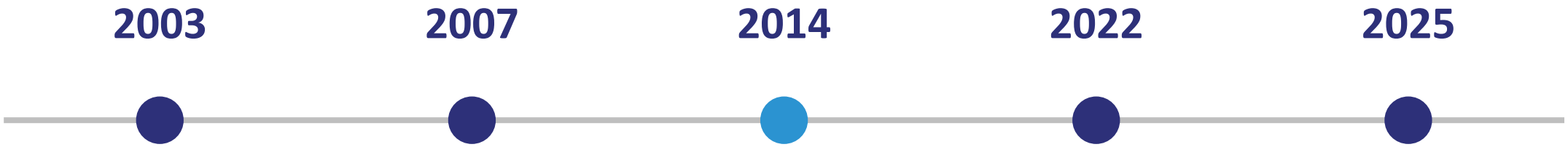
- Објектите ги креираме рачно со `new`
- Секој сервлет сам си ги прави зависностите
- Параметрите рачно ги читаме од `request`
- Конфигурација во `web.xml` / `@WebServlet`
- Посебно инсталиран Tomcat

Со Spring (од денес)

- Објектите ги креира Spring
- Зависностите се вметнуваат автоматски
- Параметрите се мапираат во Java методи
- Анотации и `application.properties`
- Вграден сервер — старт со `main()`

Spring MVC внатрешно користи сервлет (DispatcherServlet) — знаењето за сервлети останува корисно

Кратка историја



Spring

реакција на сложениот J2EE

Spring 2.5

конфигурација со анотации

Spring Boot 1.0

брз старт на апликации

Spring 6 / Boot 3

Java 17, Jakarta EE

Spring 7 / Boot 4

тековна генерација

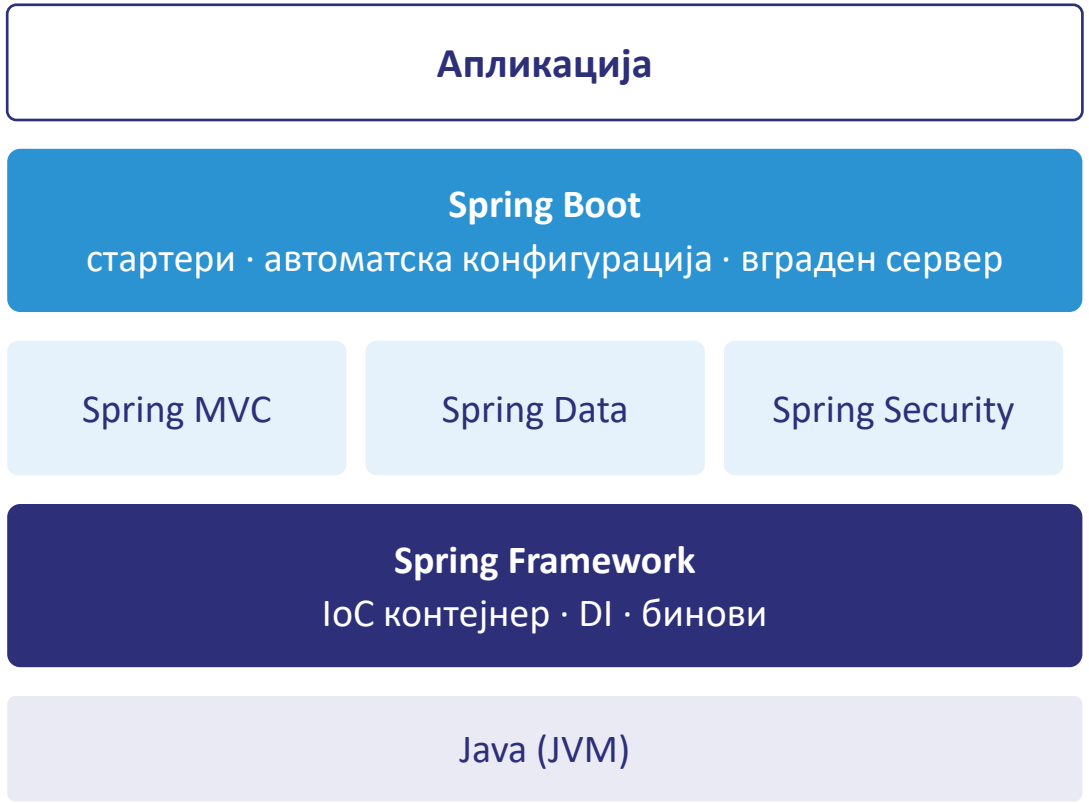
Почетна идеја: обични Java објекти (POJO) наместо тешки EJB компоненти и многу XML

Spring екосистем

Технологија	Одговорност	Едноставно објаснување
Spring Framework	IoC, DI, бинови	Ги креира и поврзува компонентите
Spring Boot	Стартување, конфигурација	Апликацијата лесно се креира и стартува
Spring MVC	HTTP барања	Ги поврзува URL адресите со Java методи
Spring Data JPA	Пристап до бази	Зачувување и читање објекти од база
Spring Security	Безбедност	Најава и контрола на пристап

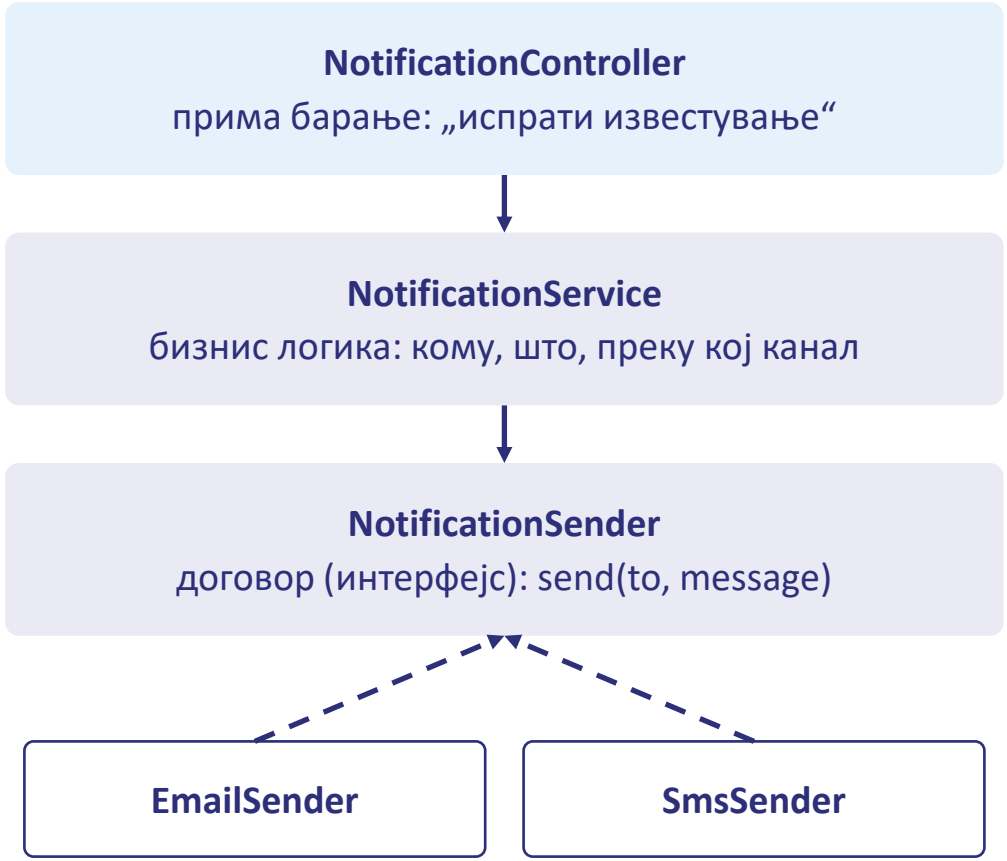
Сите модули се градат врз Spring Framework — бинови и DI се основа за сè останато

Spring Framework и Spring Boot



- Spring Boot **не е замена** за Spring Framework
- Boot = Spring + разумни стандардни поставки
- Без Boot: рачно конфигурирање на сервер, бинови, библиотеки
- Со Boot: додаваме „стартер“ и сè е подготвено
- Сè е конфигурирано (по default), но може по потреба да се додефинира

Пример: апликација за известувања



- **Час 1:** истите класи во обична Java, рачно поврзани
- **Час 2:** Spring ги креира и поврзува
- **Час 3:** `@Autowired`, повеќе имплементации, слоеви + `NotificationRepository`
- **Час 4:** singleton, `application.properties`, профили, тестови

Што е зависност?



Класата **А зависи од В** ако А ја користи В за да ја заврши својата работа.

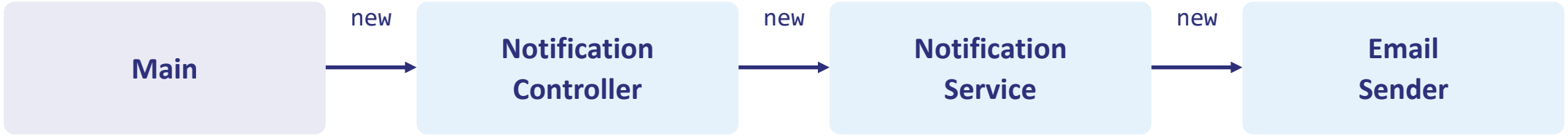
`NotificationService` **зависи од** `EmailSender`

```java
public class EmailSender {
    public void send(String to, String message) {
        System.out.println("EMAIL до " + to + ": " + message);
    }
}

public class NotificationService {
    private EmailSender sender = new EmailSender();

    public void notifyUser(String to, String message) {
        sender.send(to, message);
    }
}
```

Што се случува кога креираме контролер?



Прашање: Што ако сакаме известувања преку SMS, или да го тестираме сервисот без да се праќа вистински e-mail?

Одговор: Мора да го **менуваме кодот** на NotificationService — тој сам одлучува која зависност ја користи.

Проблем: цврсто поврзување

❌ Погрешен начин

```
public class NotificationService {

    private EmailSender sender =
            new EmailSender();   // ⚠

    public void notifyUser(String to, String message) {
        sender.send(to, message);
    }
}
```

- Сервисот **сам** ја креира зависноста
- Не може да се замени (SMS) без промена на кодот
- Тешко за тестирање — секогаш се праќа вистински e-mail
- Ако `EmailSender` има свои зависности (SMTP сервер, конфигурација), сервисот мора да знае и за нив

Решение: зависност однадвор

✅ Правилен начин

```java
public class NotificationService {

    private final EmailSender sender;

    public NotificationService(EmailSender sender) {
        this.sender = sender;
    }

    public void notifyUser(String to, String message) {
        sender.send(to, message);
    }
}
```

```java
// Main: некој друг ја креира зависноста
EmailSender sender = new EmailSender();
NotificationService service = new NotificationService(sender);
service.notifyUser("ana@finki.ukim.mk", "Добредојде!");
```

- Сервисот само **кажува** што му треба — параметар во конструкторот
- Некој друг (засега `Main`) ја **креира** и **предава** зависноста
- Полето е `final` — зависноста не може да се *„заборави“* и да не се постави

Dependency Injection

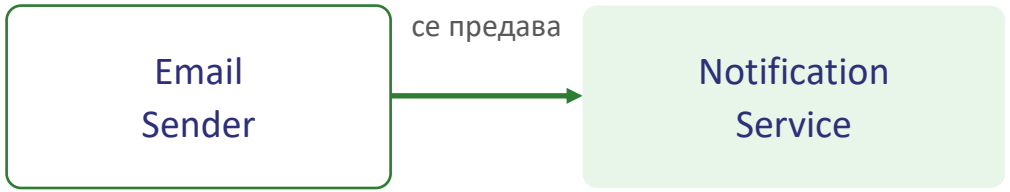
Вметнување на зависности (Dependency Injection, DI)
објектот ги **добива** зависностите однадвор, наместо сам да ги креира

Без DI

NotificationService

```
new EmailSender()
```

Со DI



- DI **не е** функционалност на Spring — тоа е општа техника. Spring само го автоматизира вметнувањето.
- Во примерот се користи **вметнување преку конструктор** (constructor injection)

Што добиваме со DI?

Флексибилност

Различна имплементација **без промена** на класата што ја користи

Пр. e-mail или SMS

Тестирање

Во тест предаваме **лажна** (fake) зависност

Без вистински e-mail, брзо

Јасност

Од конструкторот веднаш се гледа **што ѝ треба** на класата

Нема скриени new во кодот

Сите три придобивки доаѓаат од истата идеја: класата **не ги креира** своите зависности

Пример: тест со лажна зависност

```
// Лажен sender - само за тестирање
class FakeEmailSender extends EmailSender {
    String lastMessage;

    @Override
    public void send(String to, String message) {
        lastMessage = message;   // не праќа ништо
    }
}

FakeEmailSender fake = new FakeEmailSender();
NotificationService service = new NotificationService(fake);
service.notifyUser("ana@finki.ukim.mk", "Тест");

System.out.println(fake.lastMessage);   // Тест
```

- Сервисот **не знае** дека sender-от е лажен
- Ја тестираме **само** логиката на сервисот
- Можно е **само затоа што** зависноста доаѓа однадвор
- Забелешка: за DI не е задолжителен интерфејс

Повеќе имплементации: интерфејс

```
public interface NotificationSender {
    void send(String to, String message);
}

public class EmailSender implements NotificationSender {
    public void send(String to, String message) {
        System.out.println("EMAIL до " + to + ": " + message);
    }
}

public class SmsSender implements NotificationSender {
    public void send(String to, String message) {
        System.out.println("SMS до " + to + ": " + message);
    }
}
```

- Две имплементации на **ист интерфејс**
- Интерфејс воведуваме **кога има потреба** од повеќе имплементации
  - една имплементација за тестирање
  - друга имплементација за продукција
  - трета имплементација за развојна околина

Замена без промена на сервисот

```
public class NotificationService {

    private final NotificationSender sender;

    public NotificationService(NotificationSender sender) {
        this.sender = sender;
    }

    public void notifyUser(String to, String message) {
        sender.send(to, message);
    }
}
```

```
new NotificationService(new EmailSender())
        .notifyUser("ana@finki.ukim.mk", "Добредојде!");
new NotificationService(new SmsSender())
        .notifyUser("070123456", "Добредојде!");
```

Излез:

```
EMAIL до ana@finki.ukim.mk: Добредојде!
SMS до 070123456: Добредојде!
```

- `NotificationService` не се менува
- Изборот на имплементација е **надвор** од сервисот

SOLID: три принципи што ни требаат сега

S **Single Responsibility**

Секоја класа има **една** одговорност (една причина за промена)

Пример: Controller / Service / Repository

O **Open / Closed**

Ново однесување со **додавање** класа, не со менување постоечка

Пример: нов NotificationSender, сервисот не се менува

D **Dependency Inversion**

Класите зависат од **апстракции** (интерфејси), не од конкретни класи

Пример: NotificationService → NotificationSender

L (Liskov Substitution) и I (Interface Segregation) → додатен материјал на крајот

S – Една одговорност

❌ **Една класа прави сè**

NotificationServlet

- чита HTTP параметри
- одлучува преку кој канал
- праќа e-mail / SMS
- запишува во база

✅ **Секоја класа – една улога**

NotificationController
HTTP барања и одговори

↓

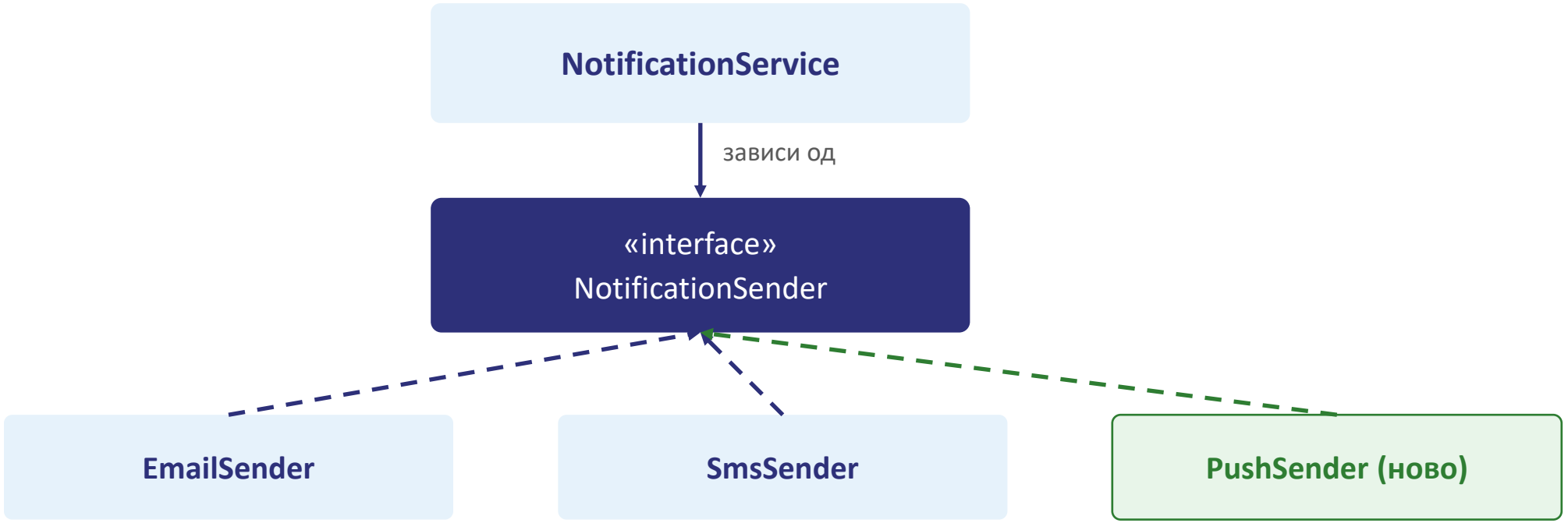
NotificationService
бизнис логика: избор на канал

↓

NotificationRepository
зачувување и читање податоци

Controller → Service → Repository: структура на речиси секоја Spring апликација

O и D во пример



- **O:** додаваме `PushSender` — `NotificationService` **не се менува**
- **D:** сервисот (високо ниво) зависи од интерфејсот, а не од `EmailSender` (ниско ниво)

Кој ги склопува објектите?

```
public class Main {
  public static void main(String[] args) {
    var repository = new NotificationRepository();
    var emailSender = new EmailSender();
    var smsSender = new SmsSender();
    var service = new NotificationService(emailSender, repository);
    var reminders = new ReminderService(smsSender, repository);
    var controller = new NotificationController(service);
    var reminderController = new ReminderController(reminders);
    // ... и така за секоја нова класа
  }
}
```

- Сите `new` се на едно место — добро!
- Но со секоја нова класа `Main` **расте**
- Мора да внимаваме на **редоследот**
- Еден или повеќе објекти од класа?

Ова го прави Spring автоматски

Комплетен код · (1/2)

NotificationSender.java

```
package com.example.notifications;

public interface NotificationSender {
    void send(String to, String message);
}
```

EmailSender.java · SmsSender.java

```
package com.example.notifications;

public class EmailSender implements NotificationSender {
    @Override
    public void send(String to, String message) {
        System.out.println("EMAIL до " + to + ": " + message);
    }
}

// ------------------------------------------------------------
package com.example.notifications;

public class SmsSender implements NotificationSender {
    @Override
    public void send(String to, String message) {
        System.out.println("SMS до " + to + ": " + message);
    }
}
```

Комплетен код · (2/2)

NotificationService.java · NotificationController.java

```java
package com.example.notifications;

public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(NotificationSender sender) {
        this.sender = sender;
    }

    public void notifyUser(String to, String message) {
        sender.send(to, message);
    }
}

// ------------------------------------------------------
package com.example.notifications;

public class NotificationController {
    private final NotificationService service;

    public NotificationController(NotificationService service) {
        this.service = service;
    }

    public void handle() {
        service.notifyUser("ana@finki.ukim.mk", "Добредојде!");
    }
}
```

Main.java

```java
package com.example.notifications;

public class Main {
    public static void main(String[] args) {
        // рачно поврзување (wiring)
        NotificationSender sender = new EmailSender();
        // NotificationSender sender = new SmsSender();

        NotificationService service =
                new NotificationService(sender);
        NotificationController controller =
                new NotificationController(service);

        controller.handle();
    }
}

// Излез:
// EMAIL до ana@finki.ukim.mk: Добредојде!
```

Резиме: од **new** до Spring

1

new во класата

Класата сама ја креира зависноста

Цврсто поврзување (tight coupling)

→

2

Рачно DI

Зависноста се предава преку конструктор

Лабаво поврзување (loose coupling), рачно склопување

→

3

Spring DI

Spring ги креира и склопува објектите

Автоматско склопување

Бинови и IoC контејнер

Прва Spring Boot апликација

Универзитет „Св. Кирил и Методиј“ во Скопје
ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

Цели

Потсетник

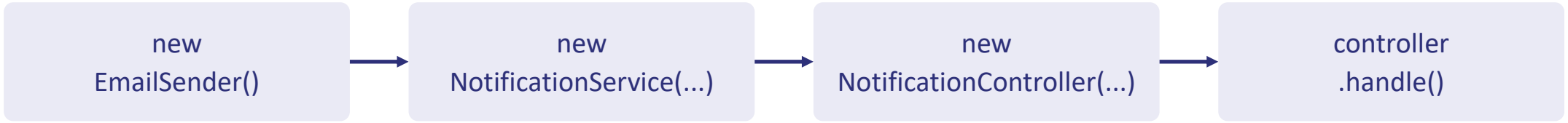
- DI = зависностите се предаваат однадвор
- Вметнување преку конструктор
- Интерфејс само кога има потреба
- Рачното склопување во `Main` брзо станува тешко

Што ќе научиме

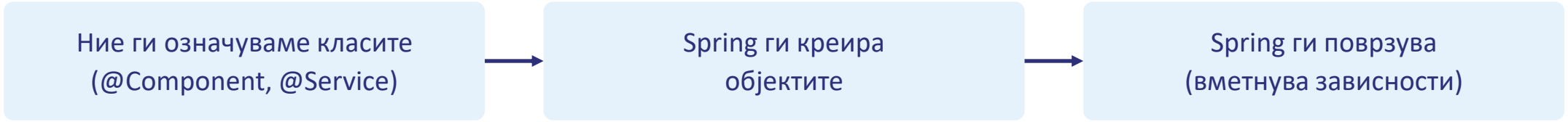
1. Што е **инверзија на контрола (IoC)**
2. Што е **бин** и што е **IoC контејнер**
3. Како Spring ги наоѓа класите: `@Component`
4. Што е **Spring Boot** и како се креира проект
5. Прва Spring Boot апликација за известувања

Инверзија на контрола (IoC)

Без IoC — ние склопуваме (Main)



Со IoC — Spring склопува



IoC: контролата за креирање и поврзување на објектите ја презема рамката Spring, не нашиот код
„Не нè барајте вие — ние ќе ве побараме“ (Hollywood principle)

IoC во кодот

Без IoC: ние склопуваме

```
// Main
var sender = new EmailSender();
var service =
        new NotificationService(sender);
var controller =
        new NotificationController(service);
controller.handle();
```

Ние **опишуваме** што постои и што на кого му треба. Spring одлучува **кога** и **како** да ги креира.

Со IoC: Spring склопува

```
@Component
public class EmailSender implements NotificationSender { ... }

@Service
public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(NotificationSender sender) {
        this.sender = sender;
    }
}
// Нема new, нема Main што склопува
```

Три основни поими

Бин (Bean)

Објект што го **креира** и со кој **управува** Spring

Пр. инстанцата од `NotificationService`

IoC контејнер

Дел од Spring што ги креира биновите, ги поврзува и управува со нивниот животен циклус

Во кодот: `ApplicationContext`

Dependency Injection

Процес на **предавање** на зависностите на бинот

Пр. Spring му го дава `EmailSender` на сервисот

Контејнерот ги **креира** биновите и преку **DI** ги **поврзува**

Што е бин, а што не е?

Бинови — ги креира Spring

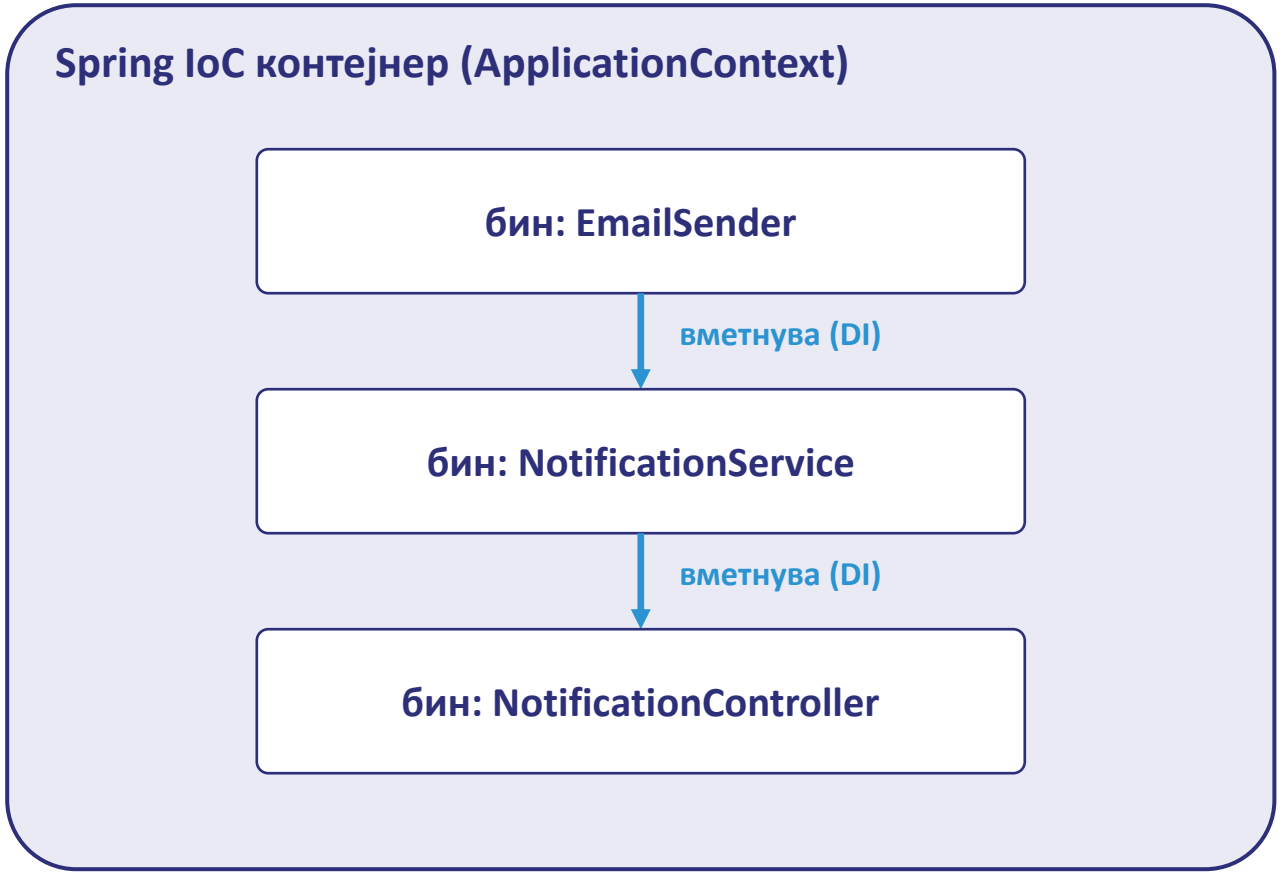
- `EmailSender` (`@Component`)
- `NotificationService` (`@Service`)
- `NotificationRepository` (`@Repository`)
- `NotificationController` (`@RestController`)
- објект вратен од `@Bean` метод

Не се бинови — обични објекти

- `Notification` — податоци за едно известување
- DTO објекти, `String`, `List`
- објекти креирани со `new` во некој метод
- сè што Spring не го креирал

Класата не е бин. Бин е **објектот** што Spring го креирал од таа класа. Бин се означува со **анотација**

IoC контејнерот



- Сервисот го **користи** sender-от
- Контролерот го **користи** сервисот
- Spring ги **креира** и **поврзува** сите три
- Секој бин се креира **еднаш***

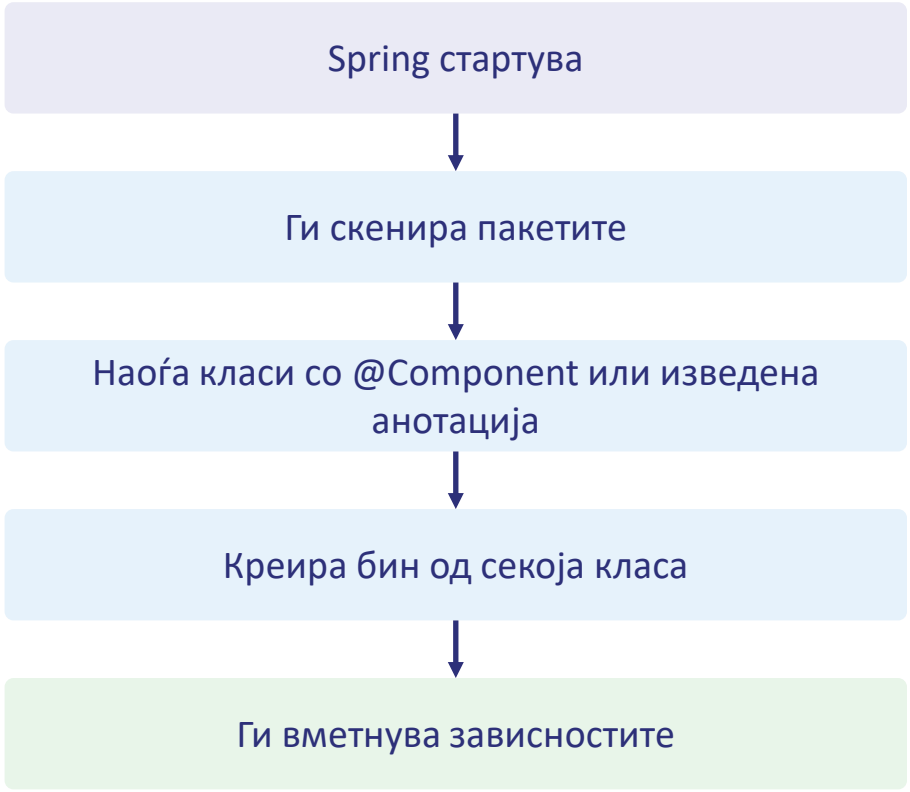
* Ова е предефинирано однесеување и може да се промени

Како Spring ги наоѓа класите?

```java
@Component
public class EmailSender implements NotificationSender {

    @Override
    public void send(String to, String message) {
        System.out.println("EMAIL до " + to + ": " + message);
    }
}
```

- @Component = „Spring, креирај бин од оваа класа“
- Процесот на барање се вика **component scan**



Анотации за бинови

Анотација	Слој / улога	Пример
`@Component`	општа компонента	EmailSender
`@Service`	бизнис логика	NotificationService
`@Repository`	пристап до податоци и надворешни системи	NotificationRepository
`@Controller`	веб слој (HTML страници)	NotificationController
`@RestController`	веб слој (JSON / текст)	NotificationController

Сите се „изведени од“ **@Component** — Spring креира бин од секоја. Различните имиња ја покажуваат **улогата** на класата.

Регистрирање бинови со @Bean

```
@Configuration
public class SenderConfig {

    @Bean
    public NotificationSender emailSender() {
        return new EmailSender();     // класата БЕЗ @Component
    }

    @Bean
    public NotificationSender smsSender() {
        return new SmsSender();
    }
}
```

- `@Configuration` — класа што дефинира бинови со анотација `@Bean`
- Секој `@Bean` **метод** враќа објект што Spring го регистрира
- Име на бинот = **име на методот**
- Корисно за класи што **не се наши** (од библиотеки)
- Ако класите се наши, можеме да ги направиме бинови преку анотација
- Ако класите не се наши, ги претвораме во бинови преку new во метод кој враќа објект од типот на класата

ApplicationContext – без Spring Boot

```
@Configuration
@ComponentScan("com.example.notifications")
public class AppConfig { }

public class Main {
  public static void main(String[] args) {
    ApplicationContext context =
      new AnnotationConfigApplicationContext(AppConfig.class);

    NotificationService service =
      context.getBean(NotificationService.class);
    service.notifyUser("ana@finki.ukim.mk", "Добредојде!");
  }
}
```

- `ApplicationContext` = IoC контејнерот на Spring
- Го креираме контејнерот и му кажуваме каде да бара класи
- `getBean()` — бараме бин од контејнерот (во пракса ретко)
- Со Spring Boot ова се случува автоматски

Component scan: кои пакети?

```
com.example.notifications
├── NotificationsApplication.java   @SpringBootApplication
├── sender
│   └── EmailSender                 ← се скенира
├── service
│   └── NotificationService         ← се скенира
└── controller
    └── NotificationController      ← се скенира

com.other
└── SmsSender                       ← НЕ се скенира
```

- Се скенира **пакетот на главната класа** и сите негови **потпакети**
- Класа надвор од скенираниот пакет не станува бин
  - Резултат: грешка при вметнување
  - Честа грешка кај почетниците!

Spring Boot – што додава?

Стартери

Една зависност наместо десетици

`spring-boot-starter-web`

Автоматска конфигурација

Ги гледа библиотеките на classpath и сам ги конфигурира потребните бинови

Вграден сервер

Tomcat е дел од апликацијата

Старт со `main()` — без инсталација

application.properties

Конфигурација на едно место

Без XML

Spring Boot = Spring + разумни стандардни поставки

Креирање проект

1. Отвори **start.spring.io** (или IntelliJ → New Project → Spring Boot)
2. Project: **Maven** · Language: **Java** · Java **17** или понова
3. Group: `com.example` · Artifact: `notifications`
4. Dependencies: **Spring Web**
5. Generate → отпакувај → отвори во IntelliJ
6. Стартувај `NotificationsApplication.main()` → http://localhost:8080

Структура на проектот

```
notifications
├── pom.xml
└── src
    ├── main
    │   ├── java/com/example/notifications
    │   │   └── NotificationsApplication.java
    │   └── resources
    │       └── application.properties
    └── test
        └── java/com/example/notifications
            └── NotificationsApplicationTests.java
```

pom.xml (дел)

```xml
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>...</version>
</parent>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

- Кај стартерот **нема верзија** — Boot ги усогласува верзиите на сите библиотеки

@SpringBootApplication

```
@SpringBootApplication
public class NotificationsApplication {

    public static void main(String[] args) {
        SpringApplication.run(
                NotificationsApplication.class, args);
    }
}
```

`SpringApplication.run()`
го креира `ApplicationContext`
го стартува вградениот Tomcat (порта 8080)

= три анотации во една:

@Configuration
класата може да дефинира бинови (преку методи анотирани со @Bean)

@ComponentScan
скенирање на пакетот и потпакетите
(скенирај ги сите компоненти анотирани со @Component и изведените од неа, и од нив креирај бинови)

@EnableAutoConfiguration
автоматска конфигурација
(креира бинови од класите на сите надворешни библиотеки што ги користи апликацијата)

Прва Spring апликација

```
@Component
public class EmailSender
        implements NotificationSender {
    public void send(String to, String msg) {
        System.out.println(
            "EMAIL до " + to + ": " + msg);
    }
}

@Service
public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(
            NotificationSender sender) {
        this.sender = sender;
    }

    public void notifyUser(String to, String msg) {
        sender.send(to, msg);
    }
}
```

```
@RestController
public class NotificationController {
    private final NotificationService service;

    public NotificationController(
            NotificationService service) {
        this.service = service;
    }

    @GetMapping("/notify")
    public String notifyUser() {
        service.notifyUser("ana@finki.ukim.mk",
                           "Добредојде!");
        return "Испратено";
    }
}
```

`@RestController` и `@GetMapping` — само за да ја стартуваме апликацијата; детално во **Spring MVC**

Што прави Spring при стартување?

1. `main()` → `SpringApplication.run()`
2. Се креира **ApplicationContext** (IoC контејнерот)
3. Component scan: ги наоѓа `EmailSender`, `NotificationService`, `NotificationController`
4. Го креира бинот `EmailSender` (нема зависности)
5. Го креира `NotificationService` и му го вметнува sender-от; потоа контролерот
6. Го стартува Tomcat на порта 8080 — апликацијата чека барања

Демо: што ако го избришеме @Component?

```
// @Component    ← го бришеме
public class EmailSender
        implements NotificationSender {
    ...
}
```

Прашање: Дали апликацијата ќе стартува?

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Parameter 0 of constructor in
com.example.notifications.service.NotificationService
required a bean of type
'com.example.notifications.sender.NotificationSender'
that could not be found.

Action:

Consider defining a bean of type
'...NotificationSender' in your configuration.
```

Како да ја прочитаме грешката?

Parameter 0 of constructor in **NotificationService** required a bean of type **NotificationSender** that could not be found.

NotificationService
кој бин не може да се креира

NotificationSender
кој бин недостасува

Најчести причини:

1. Недостасува анотација (`@Component`, `@Service`, ...)
2. Класата е надвор од скенираниот пакет
3. Се очекува `@Bean` метод кој не постои

Кој што прави во нашата апликација?

Што гледаме во кодот	Кој го обезбедува	Улога
`@Component`, `@Service`, конструктор со зависност	Spring Framework	IoC, бинови, DI
`@RestController`, `@GetMapping("/notify")`	Spring MVC	HTTP барање → Java метод
`@SpringBootApplication`, стартери, вграден Tomcat	Spring Boot	стартување и конфигурација

Spring MVC детално го учиме на посебно предавање

Резиме

- ✓ **IoC:** контролата за креирање на објектите ја презема Spring
- ✓ **Бин:** објект што го креира и со кој управува IoC контејнерот (`ApplicationContext`)
- ✓ `@Component`, `@Service`, `@Repository`, `@RestController` → класата станува бин
- ✓ **Spring Boot:** стартери, автоматска конфигурација, вграден сервер
- ✓ Еден конструктор → автоматско вметнување, без `@Autowired`

Комплетен код · (1/3)

NotificationsApplication.java · sender/NotificationSender.java

```java
package com.example.notifications;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class NotificationsApplication {
    public static void main(String[] args) {
        SpringApplication.run(NotificationsApplication.class, args);
    }
}

// ------------------------------------------------------------
package com.example.notifications.sender;

public interface NotificationSender {
    void send(String to, String message);
}
```

sender/EmailSender.java

```java
package com.example.notifications.sender;

import org.springframework.stereotype.Component;

@Component
public class EmailSender implements NotificationSender {

    @Override
    public void send(String to, String message) {
        System.out.println("EMAIL до " + to + ": " + message);
    }
}
```

Комплетен код · Час 2 (2/3)

service/NotificationService.java

```java
package com.example.notifications.service;

import com.example.notifications.sender.NotificationSender;
import org.springframework.stereotype.Service;

@Service
public class NotificationService {
    private final NotificationSender sender;

    // еден конструктор -> без @Autowired
    public NotificationService(NotificationSender sender) {
        this.sender = sender;
    }

    public void notifyUser(String to, String message) {
        sender.send(to, message);
    }
}
```

controller/NotificationController.java

```java
package com.example.notifications.controller;

import com.example.notifications.service.NotificationService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class NotificationController {
    private final NotificationService service;

    public NotificationController(NotificationService service) {
        this.service = service;
    }

    @GetMapping("/notify")        // детали: Spring MVC
    public String notifyUser() {
        service.notifyUser("ana@finki.ukim.mk", "Добредојде!");
        return "Испратено";
    }
}
```

Комплетен код · (3/3)

AppConfig.java · Main.java

```java
package com.example.notifications;

import com.example.notifications.service.NotificationService;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.example.notifications")
public class AppConfig { }

// ------------------------------------------------------------
public class Main {
    public static void main(String[] args) {
        ApplicationContext context =
                new AnnotationConfigApplicationContext(AppConfig.class);

        NotificationService service = context.getBean(NotificationService.class);
        service.notifyUser("ana@finki.ukim.mk", "Добредојде!");
    }
}

// Зависност: spring-context (без Spring Boot и без веб сервер)
// EmailSender и NotificationService — исти како на претходниот слајд
```